variables, they will not be replaced with the new values. That's why we usually prefer the volume solution.

The updated ConfigMaps or Secrets are provided to the Pod when they're mounted as volumes, but it's up to the specific application to refresh the configuration. By default, Spring Boot applications read configuration data only at startup time. There are three main options for refreshing configuration when it's provided through ConfigMaps and Secrets:

- *Rolling restart*—Changing a ConfigMap or a Secret can be followed by a rolling restart of all the Pods affected, making the applications reload all the configuration data. With this option, Kubernetes Pods would remain immutable.
- *Spring Cloud Kubernetes Configuration Watcher*—Spring Cloud Kubernetes provides a Kubernetes controller called Configuration Watcher that monitors ConfigMaps and Secrets mounted as volumes to Spring Boot applications. Leveraging the Spring Boot Actuator's `/actuator/refresh` endpoint or Spring Cloud Bus, when any of the ConfigMaps or Secrets is updated, the Configuration Watcher will trigger a configuration refresh for the affected applications.
- *Spring Cloud Kubernetes Config Server*—Spring Cloud Kubernetes provides a configuration server with support for using ConfigMaps and Secrets as one of the configuration data source options for Spring Cloud Config. You could use such a server to load configuration from both a Git repository and Kubernetes objects, with the possibility of using the same configuration refresh mechanism for both.

For Polar Bookshop, we'll use the first option and rely on Kustomize to trigger a restart of the applications whenever a new change is applied to a